

0.81.1. Ramasser une pièce : le P — elle réapparaît ailleurs

Le 0.81, mais la pièce ramassée réapparaît à la place suivante d'un tableau : on peut en ramasser sans fin.

1. Ce qu'il faut comprendre

C'est le 0.81, mais au lieu de partir hors de l'écran, la pièce **réapparaît ailleurs**.

Ce qui est nouveau ici : des places rangées dans deux tableaux, `PX` et `PY` (comme les pas du 0.32). La pièce n° `k` est en (`PX[k]`, `PY[k]`). Ramassée, on passe à la suivante : `k = (k + 1) % 5;` — après la 5e, on revient à la 1re.

On la repose aussitôt à sa nouvelle place, avec `poser`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1944 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----
```

2. UNE VARIABLE — X

```
uint8_t x = 5;           // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;           // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8
```

4. UNE VARIABLE — PX

```
uint8_t px = 15;         // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
```

5. UNE VARIABLE — PY

```
uint8_t py = 8;          // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;       // combien de pièces on a ramassées : 0 au départ

// Les places des pièces, l'une après l'autre, dans deux tableaux (le 0.32) :
//
// pièce n° :      0   1   2   3   4
// PX (colonne): 15,  3, 17,  8, 12
// PY (ligne) :   8,  2, 14,  5, 11
//
// la pièce n° 0 est en (15, 8), la n° 1 en (3, 2), la n° 2 en (17, 14)...
```

7. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PX

```
const uint8_t PX[] = { 15,  3, 17,  8, 12 };
```

8. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PY

```
const uint8_t PY[] = {  8,  2, 14,  5, 11 };
```

9. UNE VARIABLE — K

```
uint8_t k = 0;           // le numéro de la pièce EN COURS : 0, 1, 2, 3, 4, puis de nouveau 0
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {           // le A est sur la pièce (le 0.81)
            score = score + 1;               // une pièce de plus

            // La pièce SUIVANTE du tableau :
            //
            //   k = (k + 1) % 5;
            //       |         |
            //       |         +-- % 5 : le reste de la division par 5. 5 % 5 = 0 :
            //       |         après la pièce n° 4, on revient à la n° 0.
            //       +----- k + 1 : le numéro suivant
            //
            //   k : 0 → 1 → 2 → 3 → 4 → 0 → 1 ... sans fin
            k = (k + 1) % 5;

            px = PX[k];                     // la colonne de la pièce n° k...
            py = PY[k];                     // ...et sa ligne
            poser(px, py, ALPHABET[15]);    // on y pose le P : la nouvelle pièce apparaît
        }

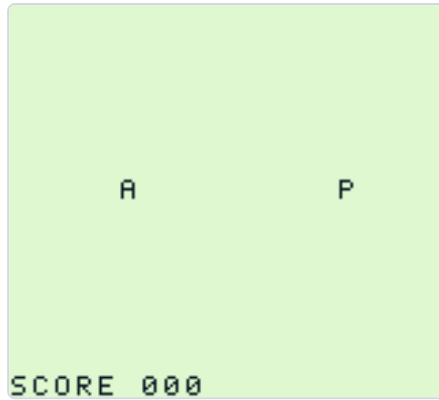
        nombre(6, 17, score);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A ramasse le P ; un autre P apparaît aussitôt ailleurs, et ainsi de suite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première pièce est ramassée
- ✓ une nouvelle pièce en (3, 2)

Ensuite : **0.81.2. Ramasser une pièce : le P — au hasard** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++